

State-of-the-Art Configurations and Best Practices for Agentic Coding with Claude Code

The deployment of autonomous, terminal-native AI coding agents marks a fundamental transition in software engineering methodologies. Unlike editor-bound artificial intelligence plugins that rely on developer-driven autocomplete paradigms, agents such as Anthropic's Claude Code operate directly within the shell environment. This execution model allows the agent to autonomously traverse file systems, execute bash commands, analyze test suite failures, and execute multi-turn refactoring loops without continuous human intervention. However, granting an autonomous process read and write access to a production codebase introduces profound challenges related to contextual accuracy, instruction adherence, and system safety.

The operational success of terminal-native agents is entirely dictated by the configuration of their environment. Context is no longer treated as a conversational prompt; it is deployed as immutable infrastructure. To achieve deterministic reliability, engineering teams must implement highly structured memory architectures, deterministic execution gates, and procedural orchestration frameworks. This comprehensive analysis evaluates the empirical research surrounding agentic context management, dissects state-of-the-art configuration boilerplates, and establishes rigorous best practices for deploying Claude Code across enterprise environments. The analysis spans persistent memory files (CLAUDE.md), OS-level lifecycle hooks (.claude/settings.json), procedural subagent orchestration (SKILL.md), and broader ecosystem interoperability standards.

The Empirical Science of Context Engineering

Early configurations for coding agents were largely driven by intuition, leading to sprawling instruction files and redundant contextual prompts. In early 2026, the software engineering community produced large-scale empirical studies that systematically dismantled widespread assumptions regarding how large language models (LLMs) process repository-level instructions. Understanding these studies is paramount for configuring Claude Code, as they quantify the exact relationship between configuration structures and agent adherence.

The Adherence Decay and the Task-Type Divergence

The most definitive evaluation of configuration file mechanics is a factorial study (arXiv:2605.10039) executed by Damon McMillan. The research captured 16,050 function-level observations across 1,650 isolated Claude Code command-line sessions, utilizing complex TypeScript codebases including Umami and a Next.js boilerplate¹. The study rigorously tested four file-structure variables that heavily populate developer advice forums: the length of the file (ranging from 25 to 500 lines), the vertical position of critical rules within the document, the

physical architecture of the files (monolithic versus nested directory structures), and the presence of deliberate internal contradictions¹.

The statistical results definitively concluded that optimizing the internal layout of a CLAUDE.md file yields absolutely no measurable impact on the agent's compliance¹. Placing a rule at the second line versus the two-hundredth line resulted in an affirmative-null Bayes factor, indicating that the position does not influence the model's attention mechanism in this context³. The mere presence of the instruction file is the primary catalyst for adherence. In control groups lacking a CLAUDE.md file, the agent followed specific tested rules 0% of the time, whereas the introduction of the file elevated compliance to 67.7%¹.

Beyond structural myths, the study revealed profound second-order implications regarding how preexisting code influences agent behavior. A massive divergence in adherence was observed based on the nature of the development task¹. When Claude Code was tasked with generating brand-new code isolated from legacy constraints, it adhered to the injected project rules 71.3% of the time¹. Conversely, when the agent was directed to modify existing functions, compliance with the explicit rules plummeted to 45.1%¹. This 26-point statistical swing demonstrates that the preexisting codebase acts as an implicit, highly localized instruction set. When a declarative rule within CLAUDE.md conflicts with the established coding habits visible in the surrounding files, the surrounding code frequently overrides the declarative instruction¹.

Furthermore, the research identified a severe temporal dilution effect. Rule compliance steadily degrades as an autonomous session progresses. The statistical model estimated a 5.6% lower odds of rule adherence for each additional function the agent generated during a continuous session ($OR = 0.944$)¹. This temporal decay proves that instruction adherence is fundamentally a context-window management problem rather than a document-formatting problem, necessitating aggressive contextual resets during long-running tasks.

The Performance Penalties of LLM-Generated Context

A parallel empirical evaluation conducted by researchers at ETH Zurich (arXiv:2602.11988) investigated the utility of utilizing the agents themselves to generate their own repository context files⁴. A common industry recommendation suggested running initialization commands (such as `/init`) to allow the LLM to scan the repository and write a comprehensive CLAUDE.md or AGENTS.md file outlining the directory structure and general purpose of the codebase⁶.

The ETH Zurich evaluation executed across 138 real-world tasks using Claude Code, Codex, and Qwen Code, revealing that this practice actively damages agent performance⁶. Relying on LLM-generated context files decreased overarching task success rates by 1.5% to 3% while simultaneously inflating inference costs by an average of 20% to 23%⁴.

The degradation in performance is attributed to the redundancy and over-retrieval characteristics of LLM-generated documentation. The generated files averaged 641 words and almost universally attempted to summarize directory structures while restating generic software engineering best practices (e.g., "write clean code", "add type hints")⁶. Because autonomous agents like Claude Code possess the native ability to execute tools like `find`, `rg`, and `ast-grep` to dynamically map directory structures, feeding them a static, text-based map forces the model to

expend critical reasoning tokens processing redundant information⁶. This mental overhead dilutes the model's focus on the actual user prompt, resulting in a higher likelihood of task failure⁷.

Even when human developers meticulously crafted the context files, the performance increase was a marginal 4%, while still incurring the 20% token cost premium⁴. The ETH researchers proved that LLM-generated files only provided a positive signal (+2.7% performance) in repositories where all existing standard documentation (such as README.md and docs/ folders) had been deliberately deleted⁹. The undeniable conclusion is that context files must abandon the goal of comprehensive documentation. They must function as surgical interventions, housing only the highly specific, non-obvious constraints that the agent cannot deduce through standard code exploration⁷.

Architecting Persistent Context via CLAUDE.md

Based on the empirical realities of context dilution and redundancy penalties, the architecture of a CLAUDE.md file must be optimized for extreme conciseness and domain-specific aberrations. The file serves as the foundational memory injected immediately following the system prompt at the initiation of every terminal session¹.

Optimization Strategies and Content Demarcation

The curation of CLAUDE.md should be governed by a strict "one-in-one-out" capacity limit. Every time a new architectural rule is appended to the document, engineers must actively seek to prune an obsolete or universally understood constraint¹. While official documentation occasionally cites a 200-line limit, this specific hard cap applies exclusively to the agent's automatic internal memory system (MEMORY.md), which truncates past 25 kilobytes. User-provided CLAUDE.md files are loaded entirely, regardless of length¹. However, optimizing for a concise 50-to-150-line file minimizes context burn and maximizes instruction adherence⁸. Table 1 delineates the specific categories of information that must be included versus those that actively harm agent efficiency through redundancy, establishing a clear framework for surgical context injection⁸.

Architectural Category	Surgical Inclusions (Mandatory)	Redundant Exclusions (Prohibited)
Toolchains & Commands	Non-standard build scripts, specific bash testing flags, bespoke linters.	Universal language norms (e.g., standard npm install instructions).
Structural Patterns	Project-specific design patterns, strict module access boundaries, required framework	File-by-file directory explanations, generic overarching codebase

	extensions.	summaries.
Quality & Styling	Deviations from default formatters, architectural gotchas, non-obvious runtime behaviors.	Subjective directives (e.g., "write efficient code", "ensure high performance").
Infrastructure	Required local environment variables, staging deployment quirks, sandboxing requirements.	Detailed API documentation (links should be provided instead).
Agent Directives	Model routing rules, specific context loading pointers (e.g., @ARCHITECTURE.md).	Rapidly changing state data (e.g., current sprint tasks, daily TODO lists).

Hierarchical Merging and Path-Scoped Constraints

Monolithic context files rapidly become a bottleneck in large-scale enterprise monorepos. To support complex architectures, Claude Code utilizes a dynamic, hierarchical merging engine. When the agent operates within a specific subdirectory, it recursively traverses up the directory tree to the repository root, concatenating every CLAUDE.md file it discovers into a unified context payload¹³. This path-scoped loading ensures that the agent receives global principles combined with local constraints.

The state-of-the-art implementation of this hierarchy relies on three distinct layers of context resolution¹³. The repository root contains the overarching file detailing global constraints, standard CI/CD actions, and universal toolchains¹³. Below the root, engineers utilize the .claude/rules/ directory to store modular markdown files containing deeply technical, domain-specific rules. These rule files employ YAML frontmatter to define glob patterns, ensuring that they activate and inject into the context window only when the agent specifically interacts with matching files¹¹. Finally, localized CLAUDE.md files are placed within isolated microservices or specialized packages. This ensures that when the agent is refactoring an independent backend service, it only loads the test commands and linting configurations relevant to that specific service, preventing the context window from being flooded by irrelevant frontend testing protocols¹³.

State-of-the-Art Boilerplate Implementations

An analysis of open-source boilerplate repositories provides concrete implementations of these theoretical configurations. These repositories demonstrate how top-tier engineering

organizations dictate behavior to autonomous agents.

The **Trail of Bits** configuration (`claude-md-template.md`) exemplifies a highly defensive, security-first posture¹⁷. The template aggressively limits the agent's autonomy regarding scope creep, explicitly forbidding the AI from executing unrequested refactoring or hypothetical future abstractions¹⁷. It mandates the use of structure-aware tooling, requiring the agent to utilize Abstract Syntax Tree search utilities (`ast-grep`) rather than simple text matching (`rg` or `grep`)¹⁷. Furthermore, it imposes severe testing constraints, barring the agent from mocking internal logic and forcing the use of advanced property-based testing (`proptest`) and mutation testing (`cargo-mutants`) to ensure the agent cannot artificially inflate test coverage with meaningless assertions¹⁷.

The **Sebastien Heyd Laravel Boilerplate** takes a highly prescriptive architectural approach¹⁸. Recognizing that LLMs often attempt to write standard framework implementations from scratch, this configuration meticulously maps out the repository's reliance on specific design patterns, including the Repository, Factory, Observer, and Command patterns¹⁸. Crucially, it provides an exhaustive manifest of custom PHP Artisan scaffolding commands (e.g., `php artisan boilerplate:datatable`). By documenting these commands, the agent is forced to use the project's native code generators rather than hallucinating custom implementations, ensuring perfect architectural alignment¹⁸.

The **Brocoders Next.js Boilerplate** mirrors this reliance on generators for React applications. It institutes a strict ban on hand-writing resource files¹⁹. Instead, the `CLAUDE.md` explicitly routes the agent to leverage a custom skill wrapper around hygen generator scripts (`npm run generate:resource`), transforming the LLM from a raw code generator into an orchestrator of deterministic, pre-approved scaffolding systems¹⁹.

The **Levu304 Polyglot Boilerplate** demonstrates how to manage multi-language rulesets within a single configuration. It explicitly maps required typing and validation boundaries across different stacks²⁰. For TypeScript backends, it enforces strict runtime validation using Zod alongside frameworks like Hono and Elysia. For React frontends, it mandates the use of Zustand or Jotai for state management, explicitly blocking the agent from defaulting to heavier legacy libraries²⁰. For Rust implementations, it enforces the use of `clippy` for strict linting and mandates the `Result` pattern for error propagation, strictly forbidding the use of `process panics`²⁰.

Table 2 contrasts the specific architectural mandates imposed by these advanced boilerplate configurations.

Boilerplate Source	Primary Ecosystem	Core Architectural Mandates	Toolchain Directives
Trail of Bits	Security / Systems	Zero scope creep; structure-aware searching; strict mocking	<code>ast-grep</code> , <code>cargo-mutants</code> , <code>prek</code> (git hooks), <code>trash</code> (replaces <code>rm</code>).

		boundaries.	
Sebastien Heyd	PHP / Laravel	Repository and Observer patterns; strict dependency on custom service providers.	php artisan boilerplate:* scaffolding commands.
Brocoders	TypeScript / React	Complete ban on hand-writing data resource files.	hygen generators (npm run generate:resource).
Levu304	Polyglot (TS, Go, Rust)	Strict runtime typing; explicit state management libraries; error propagation mandates.	Zod (TS), Zustand/Jotai (React), Clippy (Rust).

Managing the Context Lifecycle and Agent Autonomy

As empirical data proves that instruction adherence decays over the course of a multi-turn session, engineers must aggressively manage the session lifecycle and dictate specific workflows to the agent.

Mitigating Token Exhaustion

The contextual memory of the agent acts as a continuously expanding buffer, capturing conversation history, read file contents, and standard output from bash executions. As this buffer approaches the model's token limit, reasoning capabilities degrade. Proactive execution of the `/compact` command utilizes an LLM summary pass to compress the conversational history, freeing tokens while retaining high-level knowledge¹¹.

However, compaction is insufficient when transitioning between distinct functional requirements. The `/clear` command must be executed when pivoting from one feature to another, effectively zeroing out the context window. This hard reset eliminates cross-contamination of logic and resets the statistical likelihood of rule adherence back to its maximum baseline¹. Furthermore, developers must manage the size of the files they command the agent to read. Because Claude Code evaluates entire file contents to establish localized context, files exceeding 500 lines disproportionately consume token budgets and dilute focus. Enforcing a modular file architecture, restricted to 100-200 line logical units, inherently increases the precision and accuracy of the agent's edits¹¹.

The Explore, Plan, and Implement Pipeline

To prevent the agent from rushing into premature implementations that cascade into widespread architectural errors, state-of-the-art workflows enforce a strict separation of investigation and execution¹².

The pipeline initiates with the **Explore** phase, which leverages Claude's dedicated Plan mode (claude --permission-mode plan or Shift+Tab). In this mode, the agent is granted read-only access to the filesystem. It explores directory structures, executes search utilities, and maps dependencies without the ability to mutate the codebase¹². Following exploration, the agent transitions to the **Plan** phase, generating a highly structured architectural proposal detailing the files it intends to modify and the logic it intends to implement¹².

Crucially, the developer must review and validate this plan prior to execution. Empirical observations demonstrate that validating a structured plan before allowing the agent to write code reduces subsequent correction iterations by up to 60% during complex refactoring tasks¹¹. Finally, during the **Implement & Verify** phase, the prompt must explicitly demand empirical evidence of success. Rather than allowing the agent to simply assert that a fix is complete, the prompt must mandate that the agent execute the specific unit test suite and pipeline the bash exit codes and standard output into the chat context, ensuring the LLM validates its own work against deterministic testing infrastructure¹².

Deterministic Safety and Automation via Lifecycle Hooks

The most profound evolution in agentic workflows is the departure from probabilistic prompt engineering toward deterministic, OS-level execution enforcement. Instructions formatted as natural language within CLAUDE.md (e.g., "never execute rm -rf", "always format the code") are merely requests delivered to an LLM; they carry no cryptographic or system-level guarantee of compliance¹. To secure the development environment and align the agent with continuous integration standards, engineering teams implement programmatic lifecycle hooks utilizing the .claude/settings.json configuration file²².

Hooks are user-defined shell commands that execute synchronously as separate OS processes at highly specific checkpoints during the agent's execution loop. The agent daemon communicates with these hooks by piping a JSON payload detailing the current state and intended tool call into the script's standard input. The hook evaluates this payload and communicates its decision back to the agent via standard output and POSIX exit codes²³.

PreToolUse: The Programmatic Security Gate

The PreToolUse event executes immediately before Claude is permitted to trigger a system tool, such as Bash, Write, or Edit²². This event functions as a programmatic security gate, allowing developers to inspect intended commands and definitively block catastrophic operations.

If a PreToolUse hook script exits with code 2, the Claude Code daemon immediately intercepts the signal, completely cancels the pending tool execution, and injects the hook script's standard

error output directly back into the LLM's context window. This creates an automated feedback loop where the agent is informed of its specific policy violation and forced to reason through a safer alternative²². (An exit code of 0 permits execution, while an exit code of 1 represents a non-blocking diagnostic error where execution proceeds but the error is logged)²².

A standard enterprise implementation of a PreToolUse hook targets the Bash tool to prevent destructive file system operations and unsafe git mutations²². The JSON configuration extracts the intended command from standard input using jq and evaluates it against an array of prohibited regular expressions.

Table 3 outlines the data contract and standard JSON payload received by a PreToolUse hook via standard input²³.

JSON Key	Data Type	Description
hook_event_name	String	The exact lifecycle event triggering the hook (e.g., "PreToolUse").
tool_name	String	The specific system tool the agent intends to execute (e.g., "Bash", "Write").
tool_input	Object	The arguments passed to the tool (e.g., {"command": "rm -rf /"}).
cwd	String	The absolute path to the current working directory of the agent.
session_id	String	The unique identifier for the current terminal session.

By utilizing this payload, infrastructure teams construct robust security barriers. Beyond simple command blocking, advanced implementations execute secret scanning heuristics on file writes to ensure API keys are never committed, or enforce strict directory ownership boundaries, guaranteeing that an agent tasked with frontend development cannot accidentally mutate backend database schemas²⁶.

PostToolUse: Continuous Idempotent Quality Control

While PreToolUse prevents dangerous actions, the PostToolUse event ensures continuous,

idempotent quality control. This event fires immediately following the successful execution of a tool²².

Because LLMs occasionally generate code with slight syntax errors, incorrect indentation, or minor linting violations, PostToolUse hooks are deployed to execute deterministic formatters (such as prettier, black, or oxfmt) automatically every time the Write or Edit tools alter a file²². This relieves the LLM from the cognitive burden of perfect syntax formatting, allowing it to focus entirely on logical reasoning.

Furthermore, PostToolUse hooks are heavily utilized to run targeted test suites immediately following a file modification²². If an agent edits src/auth.py, a properly configured hook can automatically locate and execute tests/test_auth.py. If the pytest execution fails, the script exits with code 2, pipelining the exact stack trace back into the model's context. This transforms the agent into a closed-loop Test-Driven Development (TDD) engine, forcing it to recursively fix the regressions it introduces before it can present the final output to the user²².

Expanding the Lifecycle Event Surface

The hook system extends beyond simple tool execution, offering granular control over the entire agent session lifecycle. Table 4 categorizes the critical events available for deep orchestration²³.

Hook Event Designation	Lifecycle Trigger	Primary Architectural Application
UserPromptSubmit	Prior to the agent receiving the user's text input.	Injecting dynamic environmental context or active JIRA ticket statuses invisibly into the prompt.
SessionStart	Upon initialization of a new terminal session.	Verifying system dependencies or pulling the latest upstream git branches before work begins.
PreCompact	Immediately before the context window is summarized.	Snapshotting the exact uncompressed conversation transcript to external observability logging systems.
Stop	When the agent signals the completion of its assigned task.	Executing a final, comprehensive CI/CD validation script; blocking

		completion if critical tests fail.
SubagentStop	When a spawned, isolated subagent completes its parallel task.	Validating the delegated sub-task deliverables before merging the resulting code back to the primary branch.

It is critical to note a potential security vulnerability regarding subagents. If the agent spawns a parallel execution thread via the Agent tool, the PreToolUse and PostToolUse hooks natively execute within that subagent's scope. However, failure to implement rigorous SubagentStop validation could allow a rogue subagent to declare victory on a deeply flawed implementation, merging unstable code back to the main workspace²².

Procedural Capabilities and Subagent Orchestration via SKILL.md

While the CLAUDE.md format excels at defining static architectural facts and passive project constraints, it fundamentally fails when used to store complex, multi-step procedures. Storing detailed playbooks for deployment pipelines, pull request reviews, and production incident investigations within the global context file guarantees rapid token exhaustion and severe context dilution²⁸.

To solve the conflict between procedural knowledge and context limitations, Anthropic formalized the Claude Skills architecture. Skills represent a capability orchestration system that dynamically injects procedural instructions into the context window only when semantically triggered by the user's request²⁸.

The Progressive Disclosure Directory Structure

A skill is not a single markdown document, but a dedicated directory structure (housed in `.claude/skills/` for project-scoped skills, or `~/claude/skills/` for global user skills)²⁹. This directory architecture enables a mechanism of progressive disclosure. The agent initially ingests only the metadata of the skill; the dense procedural instructions are hidden from the context window until the skill is explicitly invoked²⁸.

A robust skill directory contains a required SKILL.md entry point, supported by isolated subdirectories. The `references/` directory houses deep technical documentation and checklists that the agent is instructed to read only if it encounters specific edge cases during execution. The `scripts/` directory stores deterministic bash executables (e.g., complex git diff parsers or log aggregators) that the skill invokes, entirely removing the burden of bash scripting from the LLM. The `examples/` directory provides few-shot templates illustrating the exact output formatting required²⁹.

YAML Frontmatter Mechanics and Invocation Control

The SKILL.md entry point is governed by specific YAML frontmatter, which dictates the skill's discovery parameters, subagent scoping, and execution permissions²⁹.

The name and description fields are mandatory and form the core of the semantic routing engine. The description must explicitly articulate the purpose of the skill and the exact phrasing or scenarios under which the agent should automatically activate it²⁹. However, allowing an LLM to automatically trigger powerful capabilities introduces severe risks. The `disable-model-invocation: true` directive is a critical safety toggle. When applied to skills handling deployments, release tagging, or git merges, it absolutely prevents the LLM from hallucinating an automatic execution; the skill will only execute when a human developer explicitly types the corresponding slash command (e.g., `/deploy`) in the terminal²⁹.

For deep investigative tasks—such as parsing extensive production logs or mapping massive dependency trees—executing the skill within the primary terminal session will overwrite the active conversational context. To prevent this, developers utilize the `context: fork` and `agent: Explore` directives within the frontmatter²¹. This instructs the system to spawn an entirely isolated subagent equipped with read-only permissions. The subagent executes the complex task in a sandboxed memory space, synthesizes the findings, and returns a compressed summary to the primary thread, flawlessly preserving the primary context window²¹.

The `allowed-tools` directive provides temporal, narrow permission management. Rather than permanently granting the agent global access to dangerous external CLIs, `allowed-tools` temporarily pre-approves the use of a specific system tool for the exact duration of the skill's invocation turn, automatically revoking the permission once the skill concludes²⁹.

The Procedural Execution Body

Beneath the YAML frontmatter, the markdown body of SKILL.md enforces rigid operational discipline. Best practices dictate a standardized structure comprising Objectives, Required Context, a numbered step-by-step Process, explicit Constraints, and Verification protocols²⁹. For example, a Pull Request Review skill utilizes argument variables (e.g., `$ARGUMENTS` or positional `$1`) to remain universally applicable across any branch²⁹. The procedural steps sequentially direct the agent to read the original issue ticket, execute a script to pull the unified git diff, trace the affected dependency graph, evaluate the code against security standards, and output a structured severity report²⁹. This transforms the LLM from an unguided assistant into a highly deterministic procedural execution engine.

However, developers must be critically aware of a specific architectural limitation regarding subagents. Official documentation reveals that the `Explore` and `Plan` subagents are hardcoded to skip the injection of `CLAUDE.md` and standard git status overviews to maximize speed and minimize token consumption²¹. Consequently, the two subagents utilized most frequently for large-scale codebase traversal are entirely blind to the global architectural rules defined in the primary memory files³⁴. If a specific design constraint is critical to an investigation, that constraint must be explicitly restated within the prompt passed to the subagent or embedded

directly within the SKILL.md body itself³⁴.

Ecosystem Interoperability and the Multi-Agent Landscape

The software engineering ecosystem in 2026 is highly fragmented. Development teams rarely commit exclusively to a single AI tool, generally operating a mixed ecosystem comprising terminal-native agents (Claude Code, Gemini CLI) alongside IDE-integrated agents (Cursor, Windsurf) and background services (Codex CLI)¹⁴. This multi-agent reality necessitates a standardized approach to context configuration to avoid catastrophic drift between proprietary rule files⁸.

Architectural Distinctions: Claude Code vs. Cursor vs. Codex

Understanding the differing architectural paradigms of these tools is critical for interoperable deployment. The primary differentiator lies in how they execute and sandbox their operations. The Codex CLI operates as a highly secure, kernel-level application utilizing Seatbelt on macOS and Landlock with seccomp on Linux³⁵. Claude Code operates at the application layer, relying heavily on the aforementioned OS-level lifecycle hooks for deterministic safety³⁵. Conversely, IDE-native agents like Cursor inherit the editor's own permission models, operating under the assumption that a human is physically at the keyboard guiding the interactive session³⁵. While Cursor excels at highly interactive, multi-pane parallel generation, Claude Code and Codex dominate unattended, shell-native continuous integration workloads³⁵.

These architectural differences extend to how context files are resolved. Cursor evaluates its `.cursor/rules/*.mdc` files conditionally per chat turn based on complex metadata matching¹⁵. Claude Code determines rule applicability based entirely on directory tree traversal and path scopes, appending matching rules as it navigates the filesystem¹⁵.

The AGENTS.md Universal Standard

To unify context injection across these disparate systems, the open-source community rallied behind the AGENTS.md standard, which is natively ingested by Cursor, Codex CLI, Aider, and numerous other platforms²⁸. The reigning best practice for enterprise interoperability is to establish AGENTS.md at the repository root as the ultimate single source of truth for all universal configurations: basic testing commands, global architectural notes, and non-negotiable security boundaries⁸.

Crucially, Claude Code does not natively read AGENTS.md⁸. If an engineering team relies exclusively on AGENTS.md, Claude Code will operate blindly. The definitive workaround, supported by Anthropic, involves creating a highly targeted CLAUDE.md file that utilizes the agent's internal path import syntax to bridge the gap¹⁵.

The primary CLAUDE.md file must simply begin with the directive: `@AGENTS.md` This one-line command forces Claude Code to load the universal agent instructions into its context window as if they were natively written in the file⁸. Beneath this import statement, developers can place

Claude-specific configurations that other agents would misinterpret, such as subagent routing preferences, dynamic context script paths (`{CLAUDE_PLUGIN_ROOT}`), and hook-based context definitions⁸.

When migrating legacy systems heavily dependent on older IDE-specific files (such as a massive, monolithic `.cursorrules` file), teams must decouple their logic. Universal guidelines should be extracted into the root `AGENTS.md`. IDE-specific UI behaviors (such as tuning inline autocomplete suggestions) should remain in localized `.cursor/rules/` files. Finally, complex procedural automations should be translated entirely into `.claude/skills/`, as static IDE rule files cannot express the deterministic bash scripting and subagent logic that skills provide⁸.

Extending System Context via Model Context Protocol (MCP)

The final layer of state-of-the-art configuration involves elevating Claude Code from an isolated codebase editor into a holistic infrastructure orchestrator. This is achieved through the Model Context Protocol (MCP), a standardized architecture that permits terminal agents to securely query remote databases, manipulate cloud provider APIs, and ingest external platform documentation dynamically¹².

Developers attach these external capabilities to Claude Code utilizing the command line interface: `claude mcp add <server-name> -- <command> [args]`¹².

Advanced engineering teams deploy highly specific MCP servers to automate complex validation pipelines. By integrating a PostgreSQL or MySQL MCP server, Claude Code gains the ability to execute dry-run schema evaluations, proving the downstream impact of ORM migrations before submitting a pull request⁴⁰. Attaching cloud infrastructure servers, such as the Terraform MCP, permits the agent to validate infrastructure-as-code manifests directly against remote staging environments⁴⁰. Furthermore, QA teams leverage testing MCPs (e.g., Playwright) to allow the agent to execute headless browser interactions, visually confirming that UI components render correctly after a React component refactoring⁴¹. This seamless integration bridges the final gap between localized source code generation and holistic, end-to-end continuous delivery automation.

Conclusion

The deployment of Claude Code and similar terminal-native autonomous agents represents a fundamental evolution in software engineering. However, the capabilities of the underlying LLM are entirely secondary to the precision of the contextual environment in which it operates. The empirical science dictates a stark departure from conversational, sprawling documentation. LLM-generated repository summaries actively harm performance, demanding instead that persistent memory files (`CLAUDE.md` and `AGENTS.md`) remain aggressively concise, surgically targeted, and rigorously pruned.

To harness the full potential of these systems, organizations must adopt a "configuration as code" paradigm for agentic workflows. By stratifying context—reserving `AGENTS.md` and `CLAUDE.md` for passive architectural constraints, leveraging `.claude/skills/` for dynamic,

sandboxed procedural workflows, and enforcing absolute, OS-level security boundaries via `.claude/settings.json` lifecycle hooks—engineering teams can safely integrate autonomous agents into production environments. Ultimately, the reliability of an autonomous coding system is defined not by the intelligence of the model, but by the deterministic rigidity of the sandbox executing it.

Bibliografia

1. CLAUDE.md Best Practices: What the Evidence Supports (2026), <https://www.alexandunlop.com/writing/claude-md-best-practices>
2. Instruction Adherence in Coding Agent Configuration Files - arXiv, <https://arxiv.org/abs/2605.10039>
3. Does Claude Code Follow CLAUDE.md? We Graded 90 Runs, <https://www.cybedefend.com/en/blog/does-claude-code-follow-claude-md>
4. Context Engineering Papers: What 20+ Studies Actually Say, <https://www.iwoszapar.com/p/agents-md-do-context-files-help>
5. Are Repository-Level Context Files Helpful for Coding Agents? - arXiv, <https://arxiv.org/abs/2602.11988>
6. Claude.md Don't Work: ETH Zurich Study Shows Context Files, <https://medium.com/@reliabledataengineering/claude-md-dont-work-eth-zurich-study-shows-context-files-reduce-success-rates-by-3-1518cac80929>
7. New ETH Zurich Study Proves Your AI Coding Agents are Failing, <https://www.marktechpost.com/2026/02/25/new-eth-zurich-study-proves-your-ai-coding-agents-are-failing-because-your-agents-md-files-are-too-detailed/>
8. What Goes in AGENTS.md (and What Doesn't) - Roland Huß, <https://ro14nd.de/what-goes-in-agents-md/>
9. Evaluating AGENTS.md: Are Repository-Level Context Files Helpful, <https://www.alphaxiv.org/abs/2602.11988v1>
10. The research is in: your AGENTS.md is probably too long, <https://developer.upsun.com/posts/ai/agents-md-less-is-more>
11. Best Practices for Claude Code | PeakLab, <https://peaklab.fr/en/blog/claude-code-best-practices>
12. Best practices for Claude Code, <https://code.claude.com/docs/en/best-practices>
13. How Claude Code works in large codebases: Best practices and, <https://claude.com/blog/how-claude-code-works-in-large-codebases-best-practices-and-where-to-start>
14. Claude Code — Starter Boilerplate - GitHub, <https://github.com/agent-anatomy/claude>
15. Cursor Rules vs CLAUDE.md Explained | Build This Now, <https://www.buildthisnow.com/blog/guide/mechanics/cursor-rules-vs-claude-md>
16. Claude Code: Practical Best Practices for Agentic Coding - Medium, <https://medium.com/@habib.mrad.83/claude-code-practical-best-practices-for-agentic-coding-2be1b62cfeff>
17. claude-code-config/claude-md-template.md at main - GitHub, <https://github.com/trailofbits/claude-code-config/blob/main/claude-md-template.md>

18. CLAUDE.md - sebastienheyd/boilerplate - GitHub,
<https://github.com/sebastienheyd/boilerplate/blob/master/CLAUDE.md>
19. CLAUDE.md - brocoders/extensive-react-boilerplate - GitHub,
<https://github.com/brocoders/extensive-react-boilerplate/blob/main/CLAUDE.md>
20. claude-code-boilerplate/CLAUDE.md at main - GitHub,
<https://github.com/levu304/claude-code-boilerplate/blob/main/CLAUDE.md>
21. Claude Code Subagents: Delegate Work to Sub-Agents - swanbase,
<https://swanbase.co/en/blog/claude-code-subagents-delegate-tasks/>
22. Claude Code Hooks Tutorial: 5 Production Hooks From Scratch,
<https://blakecrosley.com/blog/claude-code-hooks-tutorial>
23. Claude Code hooks: a practical guide (with real config) - Scalably,
<https://scalably.io/blog/claude-code-hooks-guide>
24. Claude Code settings.json Hooks: Auto-Run Scripts at Every Step,
<https://blog.vincentqiao.com/en/posts/claude-code-settings-hooks/>
25. Best Claude Code Hooks Examples and Templates in 2026,
<https://promptessor.com/blog/best-claude-code-hooks-examples-for-safer-automated-coding-workflows-in-2026>
26. Claude Code Hooks: Automation Guide | MintMCP Blog,
<https://www.mintmcp.com/blog/claude-code-hooks>
27. Claude Code Hooks: The Deterministic Control Layer for AI Agents,
<https://dotzlaw.com/insights/claude-hooks/>
28. AGENTS.md vs .cursorrules vs Claude Skills: 2026 Guide,
<https://blog.buildbetter.ai/agents-md-vs-cursorrules-vs-claude-skills-2026-comparison/>
29. <https://promptessor.com/blog/best-claude-code-skills-and-skillmd-examples-for-reusable-coding-workflows-in-2026>
30. What Are Claude Skills? How They Work, With Examples - AIPM,
<https://www.aipm-registry.com/guides/what-are-claude-skills>
31. claude-howto/03-skills/README.md at main - GitHub,
<https://github.com/luongnv89/claude-howto/blob/main/03-skills/README.md>
32. SKILL.md Format - Skills Directory Docs,
<https://www.skillsdirectory.com/docs/skill-md-format>
33. How Do You Build Your First Agent Skill? A Complete SKILL.md,
<https://agentman.ai/blog/build-your-first-agent-skill-skillmd-anatomy>
34. Two of your agents never read your CLAUDE.md - Amit Kothari,
<https://amitkoth.com/which-agents-read-claude-md/>
35. Codex CLI vs Claude Code vs Cursor: 2026 Architecture Deep-Dive,
<https://aicatchup.com/comparisons/codex-cli-vs-claude-code-vs-cursor-architecture>
36. Claude Code vs Cursor: Practical Comparison for Builders - Verdant AI,
<https://www.verdent.ai/guides/claude-code-vs-cursor-builders>
37. AGENTS.md Best Practices: Template, Examples, Guide - BetterClaw,
<https://www.betterclaw.io/blog/agents-md-best-practices>
38. The Complete Guide to Setting Up MCP Servers in 2026 - QVeris,
<https://qveris.ai/guides/claude-code-mcp>

39. MCP Server Configuration Best Practices - Stainless,
<https://www.stainless.com/mcp/mcp-server-configuration-best-practices/>
40. Harnessing Claude Code and Terraform MCP Servers for Smarter,
<https://medium.com/@pablojusue/harnessing-claude-code-and-terraform-mcp-servers-for-smarter-infrastructure-as-code-6e67f91884f1>
41. CLI Agents Part 2: Claude Code Best Practices,
<https://vld-bc.com/blog/cli-agents-part2-claude-code-best-practices>